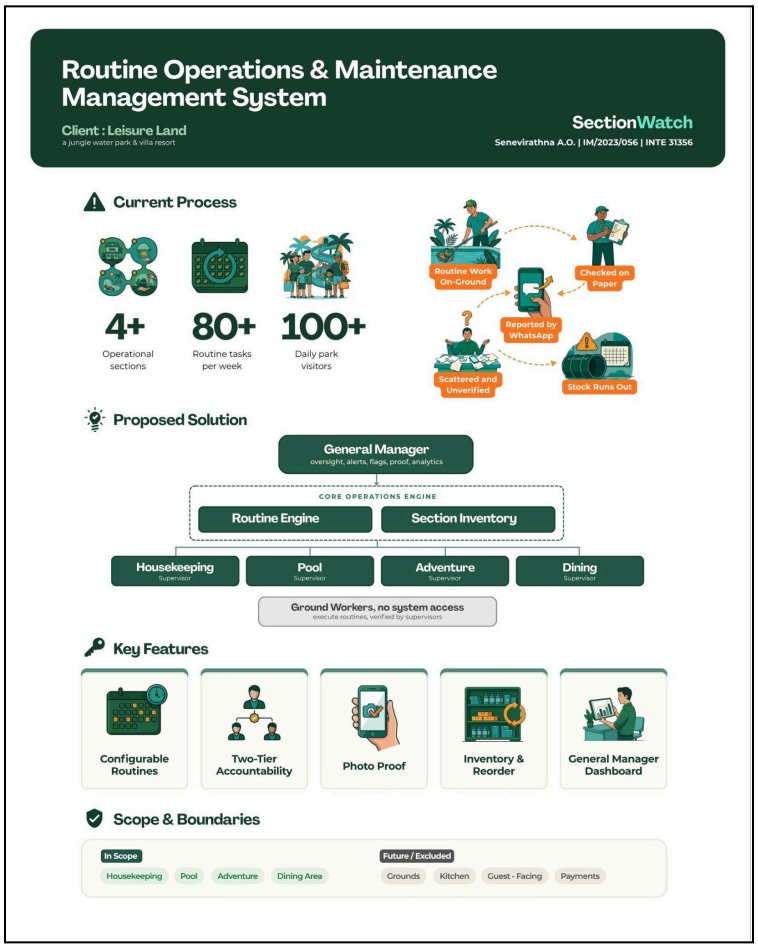
In-Class Assessment: Applying Design Patterns to a Software Project

1. Project Overview

- The name of the project for now is SectionWatch. So SectionWatch is a routine operations and maintenance management system for a jungle water park and villa resort. It covers four main operational sections (housekeeping, pool, adventure activities and dining area) The core idea is that it generates recurring tasks and tracks whether they get done properly.

- **The problem it solves :** Current proces is full manual/paper based with no reliable way to confirm routine tasks were actually completed, and there is no accountability trail, no early warning when things get missed repeatedly. My system replaces that with configurable task generation and photo-proof signoff to actually prove which tasks are done and how well they are done, and provide missed task alerts.

- **Poster:**

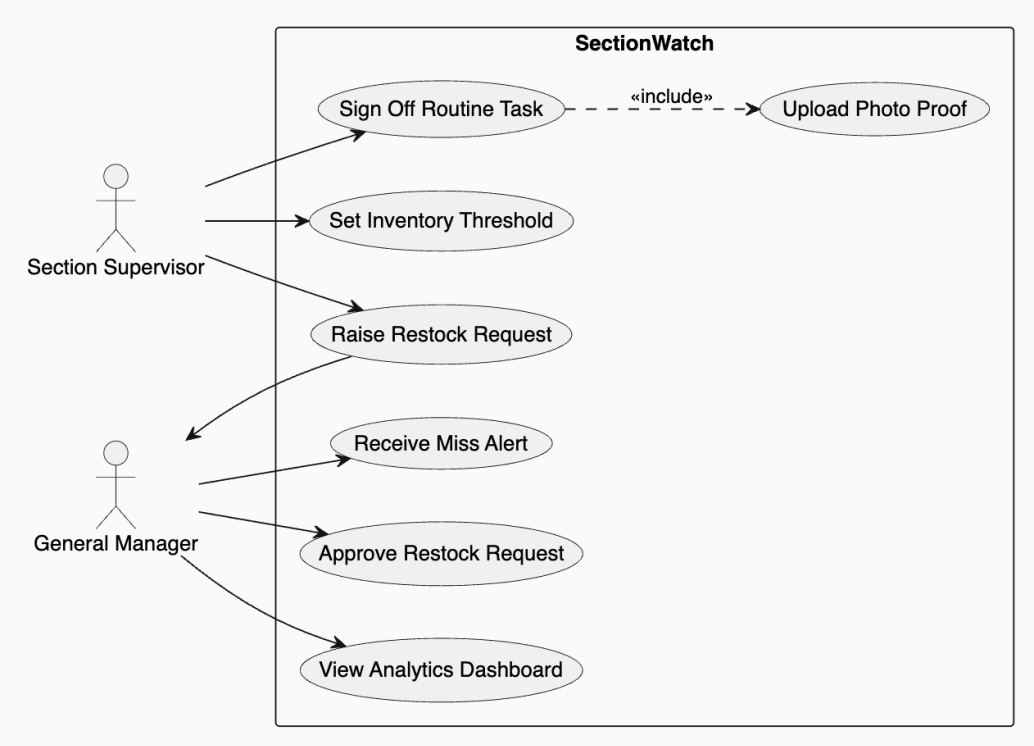


- **Main users (Stakeholders)**
  - Section Supervisors - sign off tasks with photo proof, set inventory thresholds, raise restock requests
  - General Manager (GM) - oversees via analytics dashboard, receives missed alerts and approves restock requests.
  - I'm figuring out how to connect this system with the existing front office system as i got a feedback from the poster presentation to do so. So front office managers will also be a role in this system.

2. Use Case Identification

- **Sign off routine task (Supervisor)** - supervisor completes a generated task and uploads photo proof as evidence.
- **Set inventory threshold (Supervisor)** - supervisor configures the stock level tier that triggers a restock request.
- **Raise restock request (Supervisor)** - triggered when inventory drops below threshold; goes to GM.
- **View analytics dashboard (GM)** - GM reviews completion rates, misses, and section performance.
- **Receive miss alert (GM)** - system flags missed tasks and repeat-misses to the GM.

UML Diagram -



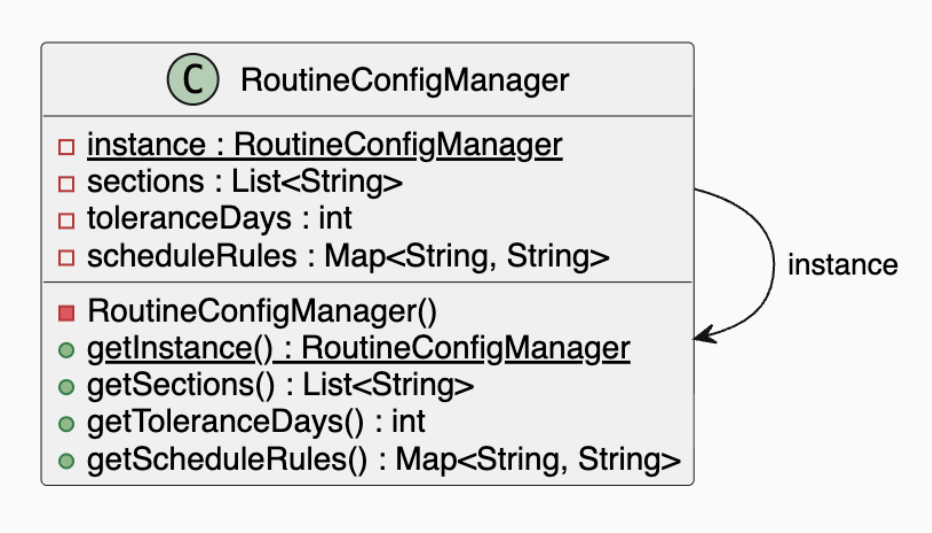
3. Applying the singleton design pattern

Scenario : In SectionWatch, the routine engine needs one central place to read its settings from. This includes the list of 4 sections, the rules for generating daily, weekly, and monthly tasks, and the tolerance window used to decide when a task is taken as missed. I apply the singleton pattern to this configuration manager, `RoutineConfigManager` so the whole system shares a single instance of these settings

Justification : singleton fits here because there should only be one set of active config values and may parts of the system need to read them. The task generator the sign off logic and the miss alert logic all depend on the same rules. If more than one instance existed, each could hold a different tolerance window or different schedule rules. That would mean the same missed task could be flagged by one part of the system and tolerated by another. That is inconsistent and hard to debug.

UML class Diagram prompt to generate diagram using PlantUML -

```
give me PlantUML for a UML class diagram of a Singleton class
called RoutineConfigManager. it should show a private static
instance field of its own type, a private constructor, a public
static getInstance method, and these fields: a list of
sections, an int toleranceDays, and a map of schedule rules,
plus getters. show the self-association arrow that singletons
have. mark static members properly.
```



Java singleton implementation prompt -

java singleton for SectionWatch, a class called RoutineConfigManager. use the static holder idiom for thread safety, not synchronized. private constructor should initialise: a list of the 4 sections (housekeeping, pool, adventure, dining), toleranceDays set to 2, and an empty map for schedule rules. give public getters for each, a getInstance, and a short main that gets the instance twice and prints whether both references point to the same object. keep it plain java, no frameworks.

Implementation generated with AI -

```
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class RoutineConfigManager {

    // Instance fields: the routine engine's configuration
    private final List<String> sections;
    private final int toleranceDays;
    private final Map<String, String> scheduleRules;

    // Private constructor prevents external instantiation
    private RoutineConfigManager() {
        this.sections = Arrays.asList("housekeeping", "pool",
"adventure", "dining");
        this.toleranceDays = 2;
        this.scheduleRules = new HashMap<>();
    }

    // Static holder idiom: the instance is created only when
    // getInstance() is first called, and the JVM guarantees
the
    // class-loading is thread-safe without needing
synchronized.
    private static class ConfigHolder {
        private static final RoutineConfigManager INSTANCE =
new RoutineConfigManager();
    }

    public static RoutineConfigManager getInstance() {
        return ConfigHolder.INSTANCE;
    }
```

```
    // Getters
    public List<String> getSections() {
        return sections;
    }

    public int getToleranceDays() {
        return toleranceDays;
    }

    public Map<String, String> getScheduleRules() {
        return scheduleRules;
    }

    // Demonstration
    public static void main(String[] args) {
        RoutineConfigManager config1 =
RoutineConfigManager.getInstance();
        RoutineConfigManager config2 =
RoutineConfigManager.getInstance();

        System.out.println("Sections: " +
config1.getSections());
        System.out.println("Tolerance window (days): " +
config1.getToleranceDays());
        System.out.println("Same instance? " + (config1 ==
config2));
    }
}
```

Output :

```
[Running] cd "/Users/osandasenevirathna/Downloads/" && javac RoutineConfigManager.java && java RoutineConfigManager
Sections: [housekeeping, pool, adventure, dining]
Tolerance window (days): 2
Same instance? true
```

4. Applying the Factory Method Design Pattern

Scenario : Routine engine generating task instances of different task type (Daily, Weekly, Monthly). So instead of hardcoding the creation of these task objects inside the engine, I apply the factory method pattern. There is a common RoutineTask type that all task instances share and separate creator classes for each frequency (

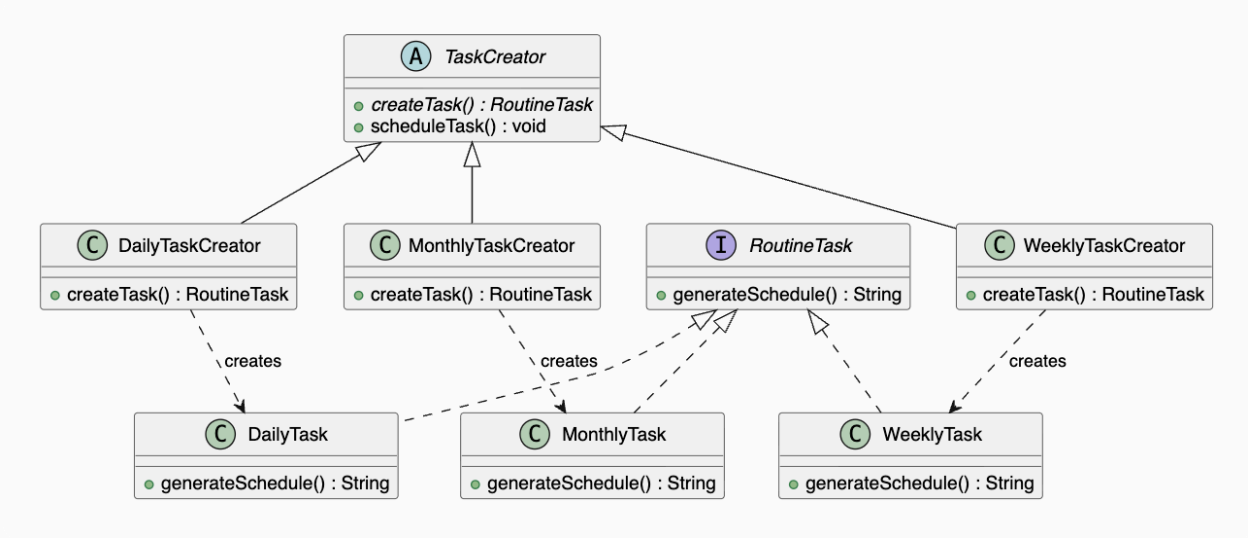
DailyTaskCreator, WeeklyTaskCreator, MonthlyTaskCreator) that are responsible for building their own kind of task. The engine asks for a creator for a task without needing to know exactly which concrete class it gets back.

Why it is suitable : Factory method fits here because the routine engine needs to create task objects, but the code that triggers task generation should not be tied to the specific task type classes. If a big if/else or switch was used on the task type, everytime i add a new type of task, i would have to go back and edit the same block which increases the risk of breaking parts that already work. With factory method, adding a new task means writing only new task class, and one new creator class, without touching the existing creation code.

UML Class diagram prompt to get PlantUML Code:

```
give me PlantUML for a UML class diagram showing the Factory
Method pattern for SectionWatch. product interface RoutineTask
with a method generateSchedule(). three concrete products
DailyTask, WeeklyTask, MonthlyTask implementing it. abstract
creator class TaskCreator with an abstract factory method
createTask() returning RoutineTask. three concrete creators
DailyTaskCreator, WeeklyTaskCreator, MonthlyTaskCreator each
extending TaskCreator and creating their matching task. show
the implements arrows, the extends arrows, and a dashed
dependency from each concrete creator to the product it
creates. keep it clean.
```

Generated Dlagram :



Java Code generated with AI -

```
// ===== Product interface =====
interface RoutineTask {
    String generateSchedule();
}

// ===== Concrete products =====
class DailyTask implements RoutineTask {
    @Override
    public String generateSchedule() {
        return "Daily task: runs every day, once per 24
hours.";
    }
}

class WeeklyTask implements RoutineTask {
    @Override
    public String generateSchedule() {
        return "Weekly task: runs once per week on a set day.";
    }
}

class MonthlyTask implements RoutineTask {
    @Override
    public String generateSchedule() {
        return "Monthly task: runs once per month on a set
date.";
    }
}

// ===== Abstract creator =====
abstract class TaskCreator {

    // The factory method - subclasses decide which product to
create
    protected abstract RoutineTask createTask();

    // Concrete method that USES the product without knowing
its concrete type.
    // This is the behaviour that demonstrates the point of
Factory Method.
    public void scheduleTask() {
        RoutineTask task = createTask();
        System.out.println(task.generateSchedule());
    }
```

```
}

// ===== Concrete creators =====
class DailyTaskCreator extends TaskCreator {
    @Override
    protected RoutineTask createTask() {
        return new DailyTask();
    }
}

class WeeklyTaskCreator extends TaskCreator {
    @Override
    protected RoutineTask createTask() {
        return new WeeklyTask();
    }
}

class MonthlyTaskCreator extends TaskCreator {
    @Override
    protected RoutineTask createTask() {
        return new MonthlyTask();
    }
}

// ===== Demonstration =====
public class FactoryMethodDemo {
    public static void main(String[] args) {
        TaskCreator[] creators = {
            new DailyTaskCreator(),
            new WeeklyTaskCreator(),
            new MonthlyTaskCreator()
        };

        for (TaskCreator creator : creators) {
            creator.scheduleTask();
        }
    }
}
```

Output :

```
[Running] cd "/Users/osandasenevirathna/Downloads/" && javac FactoryMethodDemo.java && java FactoryMethodDemo
Daily task: runs every day, once per 24 hours.
Weekly task: runs once per week on a set day.
Monthly task: runs once per month on a set date.

[Done] exited with code=0 in 0.391 seconds
```

5. Applying the Facade Design Pattern

Scenario : In SectionWatch, signing off a task is not a single action. When a supervisor marks a task done and uploads a photo, several things have to happen in order.

The photo proof gets validated → the task gets recorded as complete → the system checks if it was done within the tolerance window → the GM's Dashboard gets updated → a miss alert is sent if the task was late

Each of these steps live in its own class, that is five classes the would need to know about and call in the right order. I used the facade pattern to hide all that. The facade class TaskSignOffFacade gives one method, completeSignOff(), that runs the whole sequence behind the scenes.

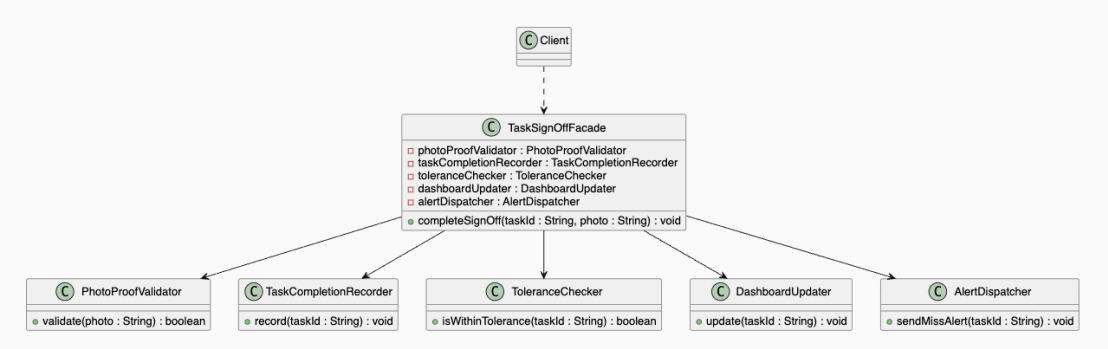
Why it improves usability : The caller only deals with one method now. It doesn't need to know the five classes behind it, the correct order to call them in or the rules between steps, like tolerance can be checked after completion is recorded.

If the task complete process changes lanter, I only edit the facade. The rest of the code stays the same. This keeps the UI layer loosely coupled from the sub system and makes the task completion flow easy to use and easy to maintain.

UML diagram prompt to generate PlantUML Code -

```
give me PlantUML for a UML class diagram of the Facade pattern
for SectionWatch. a facade class TaskSignOffFacade with a
method completeSignOff(taskId, photo). behind it, five
subsystem classes: PhotoProofValidator with validate(),
TaskCompletionRecorder with record(), ToleranceChecker with
isWithinTolerance(), DashboardUpdater with update(),
AlertDispatcher with sendMissAlert(). the facade holds a
reference to each subsystem class, show solid association
arrows from the facade to all five. also show a Client class
with a single dependency arrow pointing only at the facade.
keep it clean.
```

Generated Diagram from PlantUML-



Java Code generated with AI -

```
// ===== Subsystem classes =====

class PhotoProofValidator {
    public boolean validate(String photo) {
        System.out.println("Validating photo proof: " + photo);
        // A real check would inspect the file; here we treat a
non-empty photo as valid.
        return photo != null && !photo.isEmpty();
    }
}

class TaskCompletionRecorder {
    public void record(String taskId) {
        System.out.println("Recording task " + taskId + " as
complete with timestamp.");
    }
}

class ToleranceChecker {
    public boolean isWithinTolerance(String taskId) {
        System.out.println("Checking task " + taskId + "
against the tolerance window.");
        // A real check would compare completion time to the
config's tolerance window.
        return true;
    }
}

class DashboardUpdater {
    public void update(String taskId) {
        System.out.println("Updating GM dashboard for task " +
taskId + ".");
    }
```

```
}

class AlertDispatcher {
    public void sendMissAlert(String taskId) {
        System.out.println("Sending miss alert to GM for task "
+ taskId + ".");
    }
}

// ===== Facade =====

class TaskSignOffFacade {

    private final PhotoProofValidator photoProofValidator;
    private final TaskCompletionRecorder
taskCompletionRecorder;
    private final ToleranceChecker toleranceChecker;
    private final DashboardUpdater dashboardUpdater;
    private final AlertDispatcher alertDispatcher;

    public TaskSignOffFacade() {
        this.photoProofValidator = new PhotoProofValidator();
        this.taskCompletionRecorder = new
TaskCompletionRecorder();
        this.toleranceChecker = new ToleranceChecker();
        this.dashboardUpdater = new DashboardUpdater();
        this.alertDispatcher = new AlertDispatcher();
    }

    // One simple entry point that hides the whole multi-step
workflow.
    public void completeSignOff(String taskId, String photo) {

        // Step 1: photo must be valid, otherwise stop the
whole process.
        if (!photoProofValidator.validate(photo)) {
            System.out.println("Sign-off rejected: invalid
photo proof.");
            return;
        }

        // Step 2: record the task as complete.
        taskCompletionRecorder.record(taskId);
```

```
        // Step 3: check whether it was done inside the
tolerance window.
        boolean withinTolerance =
toleranceChecker.isWithinTolerance(taskId);

        // Step 4: update the GM dashboard.
        dashboardUpdater.update(taskId);

        // Step 5: only alert if the task fell outside
tolerance.
        if (!withinTolerance) {
            alertDispatcher.sendMissAlert(taskId);
        }

        System.out.println("Sign-off complete for task " +
taskId + ".");
    }
}

// ===== Demonstration =====

public class FacadeDemo {
    public static void main(String[] args) {
        TaskSignOffFacade facade = new TaskSignOffFacade();
        facade.completeSignOff("HK-101",
"housekeeping_photo.jpg");
    }
}
```

Output :

```
[Running] cd "/Users/osandasenevirathna/Downloads/" && javac FacadeDemo.java && java FacadeDemo
Validating photo proof: housekeeping_photo.jpg
Recording task HK-101 as complete with timestamp.
Checking task HK-101 against the tolerance window.
Updating GM dashboard for task HK-101.
Sign-off complete for task HK-101.

[Done] exited with code=0 in 0.338 seconds
```

6. Reflection and Comparison

Benefits :

- Singleton gave SectionWatch one shared config manager**.** Every part of the routine engine reads the same sections, schedule rules and tolerance window, so the settings won't get split
- Factory method let the routine engine create daily, weekly and monthly tasks without hardcoding the choice. A new frequency can be added later by writing one new class without touching the existing code.
- Facade gave the task completed workflow a single entry point. The caller runs five step process through one method and doesn't need to know the subsystem behind it

Comparision of Roles :

- Use singleton when there must be exactly one instance of something shared globally
- Use factory method when you need to create objects from a family that shares an interface, and the exact type is decided at runtime.
- Use facade when a subsystem is complex and you want to give the caller a simpler way to use it

7. Generative AI Usage :

- **How AI was used :**
  - It was used as a supporting tool in two ways. First to help structure the written answers, deciding how to present each design pattern scenario and the justification points to cover. Secondly to generate UML diagrams and Java implementations for the three patterns based on the specific context of our system I used the help of AI.

    The prompts used were given above within the parts where it was used.

- **How output was verified :**
  - The use case diagram had a syntax error, which was fixed. The restock to GM relationship was checked against the usecase notation rules.
  - All java code was compiled and run to confirm it works